Carlos Daniel Martinez García
Douglas Daniel Pérez Hernández

Delivery 5: FinOps Optimization

Task #2: Refactoring and “Before vs. After” Comparison

Muy bien la idea aquí es mejorar el rendimiento sin romper funcionalidad, y luego volver a correr exactamente los mismos benchmarks para demostrar la mejora y ejemplificar que realmente al refactorizar todos los hotspots podemos obtener grandes mejoras para nuestro proyecto.

El archivo clave es:

conduit/apps/articles/views.py

El problema principal está en cómo se cargan las relaciones del modelo Article.

Actualmente tenemos algo como esto:

queryset = Article.objects.select_related('author', 'author__user')

Esto solo trae:

- author
- author.user

Pero el serializer también usa otras relaciones:

- tags
- favorited_by
- favoritos
- profile data

Entonces Django termina haciendo consultas adicionales por cada artículo lo que genera mucho delay al momento de obtener resultados.

Este es el típico problema **N+1 querys.**

Veamoslo de la siguiente forma:

Supongamos que tenemos 2,000 artículos y el serializer necesita cargar algo extra para cada uno:

1 query principal

+ 2000 queries adicionales

Carlos Daniel Martinez García
Douglas Daniel Pérez Hernández

El resultado que obtenemos es de 2,001 querys, esto es lo que está volviendo lento nuestro proyecto, ya que si lo imaginamos a una escala mayor con muchísimos más usuarios y todos ingresando a este módulo en simultáneo, es un caos.

La refactorización de esta parte, por pequeña y sencilla que parezca nos ayuda a poder reducir todos estos tiempos perdidos, en este caso vamos a utilizar: **prefetch_related().** Esto hace que Django cargue las relaciones en conjunto, no una por una.

Pasamos de tener este código:

```
queryset = Article.objects.select_related('author', 'author__user')
```

A tener este otro:

```
queryset = Article.objects.select_related(
    'author',
    'author__user'
).prefetch_related(
    'tags',
    'favorited_by'
)
```

¿Por qué esto mejora el performance?

Antes teníamos esto:

1 query articles

+ **N querys tags.**
+ **N querys favorites.**

Pero ahora lo que tenemos es:

1 query articles.

1 query tags.

1 query favorites.

Si continuamos con el ejemplo que se dió anteriormente de los 2000 datos, pasaremos de tener 2001 querys a solamente tener 3. Esto reduce muchísimo el tiempo al realizar las consultas en la base de datos.

Carlos Daniel Martinez García
Douglas Daniel Pérez Hernández

Entonces, realizamos una prueba unitaria secuencial y los resultados obtenidos fueron los siguientes:

```
PS C:\Users\danie\Downloads\Github\django-realworld-example-app> Measure-Command { Invoke-WebRequest http://localhost:8000/api/articles -UseBasicParsing }

Days              : 0
Hours             : 0
Minutes           : 0
Seconds           : 0
Milliseconds      : 97
Ticks             : 972714
TotalDays         : 1.125826388888889E-06
TotalHours        : 2.70198333333333E-05
TotalMinutes      : 0.00162119
TotalSeconds      : 0.0972714
TotalMilliseconds : 97.2714
```

Luego realizamos una prueba de concurrencia ya con 100 usuarios otra vez:

```
Load test finished
-------------------
Users simulated: 100
Total test time (s): 9.9885087
Min response time (ms): 59.5757
Avg response time (ms): 68.931393
Max response time (ms): 105.8028
```

Prueba Unitaria			
	Antes (Sin Optimizar)	**Después (Optimizado)**	**Impacto / Mejora**
Código ORM (Django)	Solo select_related	select_related + prefetch_related	Solución al problema de N+1 consultas.
Prueba Unitaria (1 solicitud)	10.041 segundos	0.097 segundos	~99% de reducción en el tiempo de respuesta.

Carlos Daniel Martinez García
Douglas Daniel Pérez Hernández

Prueba de Carga (100 usuarios)			
	Antes (Sin Optimizar)	**Después (Optimizado)**	**Impacto / Mejora**
⏱️ *Tiempo total del test*	29.34 segundos	9.98 segundos	~3x más rápido en procesar toda la carga.
📉 *Tiempo promedio (Avg)*	222.57 ms	68.93 ms	~69% de mejora en el tiempo promedio.
🛑 *Tiempo máximo (Max)*	497.92 ms	105.80 ms	~78% de mejora en los picos de latencia.

La decisión de refactorizar el ORM de Django incorporando `prefetch_related` (para las relaciones de muchos a muchos como `tags` y `favorited_by`) fue una medida crítica para erradicar el problema de las consultas N+1 que estaba saturando nuestra base de datos. Como demuestran las métricas, consolidar la recuperación de datos en unas pocas consultas optimizadas redujo el tiempo de respuesta promedio en un 69% bajo estrés y aceleró la capacidad total de procesamiento en casi tres veces. Esta optimización no solo eliminó un cuello de botella severo que afectaba el rendimiento de una sola solicitud (reduciendo el tiempo en un 99%), sino que garantiza que la API sea robusta, estable y capaz de escalar eficientemente frente a un tráfico real de usuarios concurrentes, mejorando drásticamente la experiencia final del cliente.